

Sentence-BERT を用いた用途・予算別 PC 構成推薦システム 付録資料

学籍番号 24026111, 氏名佐藤信晃

2026 年 8 月 1 日

1 LLM の利用記録

本課題では、LLM を以下の目的で利用した。

- 課題文の要件整理と、既存プロトタイプとの差分確認
- 文字化けおよび Python 構文エラーの修正
- 入力検証, キャッシュ, 性能評価処理の実装支援
- レポート本文の構成と表現の推敲

主な指示は「NLPreport.md の要件に沿って、既存の PC パーツ推薦プロトタイプを完成させること」であった。LLM から得た提案はそのまま採用せず、Python の構文検査, 依存ライブラリのインポート, Sentence-BERT による実検索, 10 件の一括評価を実行して確認した。

1.1 対話記録

以下では、実際に LLM へ入力した指示の内容を時系列で示す。対話時の短い口語表現は、意図を変えない範囲でレポート用の文章に要約した。したがって、以下の記述は逐語的な対話ログではなく、指示内容の要約である。各指示に対する応答の要点、実際に採用した内容、確認方法も併記する。

対話 1: 課題要件と既存実装の確認

ユーザの指示内容 (要約)

「NLPreport.mdに記載された課題要件を確認し、既存の PC パーツ推薦システムを課題に適合するプロトタイプへ修正する。」

LLM の応答・作業

課題文, 既存の app.py, pc_builds.csv, requirements.txt を読み、課題が Sentence-BERT または gensim を必ず利用する NLP プロトタイプの設計・実装・評価であると整理した。既存コードは日本語の文字化けに加え、文字列の引用符が途中で失われており、Python として実行できない状態であると指摘した。

採用内容と確認

PC 構成推薦という題材は課題条件を満たすため維持し、文字化けした UI 文言と構文を修正した。python -m py_compile app.py による構文検査と, Streamlit, pandas, sentence-transformers, torch のインポート確認を行った。

対話 2：システムのゴールの明確化

ユーザの質問内容（要約）

「PC 構成推薦システムの対象ユーザ，入出力，Sentence-BERT の利用箇所，および性能評価の方法を整理する。」

LLM の応答

利用者が自然文で PC 用途と予算を入力すると，Sentence-BERT で用途の意味を比較し，予算内の PC 構成を推薦するシステムを完成形として説明した。また，プロトタイプの作成だけでなく，Top-1・Top-3 正解率による評価と，指定形式の本文 PDF および付録 PDF が必要であると整理した。

採用内容

この説明を基に，実装では「構成を検索」と「性能評価」の二つの画面を用意し，レポートでは課題設定，技術概要，設計，実装，考察の順で説明することにした。

対話 3：推薦処理と評価処理の改善

ユーザの指示内容（要約）

「Sentence-BERT を利用した推薦処理を実装し，プロトタイプの性能を比較可能な指標で評価できるようにする。」

LLM の提案

検索のたびに全候補をベクトル化すると処理が重複するため，モデル，CSV データ，全候補の埋め込みをキャッシュする構成が提案された。また，空の入力や予算内候補がない場合の処理，登録文をそのまま使わない 10 件の言い換え評価が追加された。

採用内容と確認

`st.cache_resource`でモデルとコーパス埋め込みを，`st.cache_data`で CSV をキャッシュした。Sentence-BERT モデルを実際に読み込み，「WQHD でゲームを快適に遊びたい」という入力に対して「WQHD ゲーム」が類似度 0.883 で 1 位になることを確認した。さらに 10 件の一括評価を実行し，Top-1，Top-3 とともに 10/10 件となった。

対話 4：レポートテンプレートへの適用

ユーザの指示内容（要約）

「`NLP_template_UTF8.tex`をレポートのテンプレートとして使用し，実装内容と評価結果をレポート用の文章にまとめる。」

LLM の応答・作業

テンプレートの指定 section 名を変更しないという課題要件を確認し，「課題設定」「技術概要」「システムの設計」「プロトタイプの実装」「最終的なシステムの考察」の五章で本文を作成した。Sentence-BERT の仕組み，コサイン類似度の数式，処理フロー，関数構成，評価方法，制約と改善案を追加した。

確認

参考文献には Sentence-BERT の原論文，使用モデルの Model Card，Sentence Transformers 公式文書を記載した。ソースコード全文を本文に載せず，推薦処理の中核部分だけを掲載した。

対話 5：LaTeX から PDF への変換

ユーザの指示内容（要約）

「作成した LaTeX 文書をコンパイルし，提出内容を確認できる PDF を生成する。」

LLM の応答・作業

実行環境に pLaTeX がなかったため，Tectonic をプロジェクト内へ導入した。元テンプレートの `jsarticle`と `dvipdfmx`指定は Tectonic の XeTeX 処理に直接対応しなかったため，本文内容を維持したまま `article`，`xeCJK`，Windows の日本語フォントを使う設定へ変更した。日付はテンプレートと同様に `\date{\today}`を使い，「年・月・日」の日本語表記になるよう再定義した。

確認

コンパイル終了コード、PDF ヘッダ、ファイル終端を確認し、正常な PDF ファイルが生成されたことを確認した。

対話 6：実行画面と結果の追加

ユーザの指示内容（要約）

「プロトタイプの実行画面をレポートへ掲載し、表示された推薦結果をレポート用の文章と表で説明する。」

LLM の応答・作業

利用者が用意した二枚の実行画面を本文の実装章へ掲載した。画面に表示された入力条件、三つの推薦候補、類似度、価格、予算残額を表として整理した。単に画像を貼るだけでなく、第1候補が入力用途と対応した理由、第2候補が低予算 e スポーツとなったことから分かる意味検索の限界を考察した。

確認

画像をレポート用の英数字ファイル名で `figures` ディレクトリへ保存し、本文から図番号で参照した後、PDF を再コンパイルした。

対話 7：本文 PDF と付録 PDF の分離

ユーザの指示内容（要約）

「課題の提出形式に従い、レポート本文と付録資料を別々の PDF ファイルに分離する。」

LLM の応答・作業

課題文では本文 PDF と付録 PDF を別ファイルで提出すると指定されていたため、本文末尾に含めていた LLM 利用記録、ソースコード、評価結果の生データを削除し、新しい付録用 TeX ファイルへ移動した。

確認

本文 PDF と付録 PDF を個別にコンパイルし、本文側には指定された五章と参考文献だけが、付録側には補助資料だけが含まれることを確認した。

1.2 LLM 出力の扱い

LLM の提案には、実装環境に存在しないコマンドや、LaTeX 処理系によって利用できない設定が含まれる可能性がある。そのため、生成されたコードや文章を無条件には採用せず、エラーメッセージを確認して修正し、実行結果を根拠として採否を判断した。特に推薦精度 100% という結果については、評価数が 10 件と少なく、開発者が作成したデータであるため、一般的な精度を保証しないことを本文の考察に明記した。

2 ソースコード

作成したソースコードは次のファイルから構成される。

- `app.py` : Streamlit 画面、推薦処理、評価処理
- `pc_builds.csv` : 26 件の PC 構成コーパス
- `requirements.txt` : 実行に必要なライブラリ

実行コマンドを以下に示す。

```
.\venv\Scripts\python.exe -m streamlit run app.py \
--server.fileWatcherType none
```

2.1 推薦処理の主要部分

```
def recommend(query: str, budget: int, top_k: int = 3):
    query = query.strip()
    if not query or budget <= 0 or top_k <= 0:
        return []

    all_rows = load_data()
    eligible_indices = [
        index
        for index, row in enumerate(all_rows)
        if int(row["price_yen"]) <= budget
    ]
    if not eligible_indices:
        return []

    model = load_model()
    corpus_embeddings = load_corpus_embeddings()
    query_embedding = model.encode(
        query,
        convert_to_tensor=True,
        normalize_embeddings=True,
    )
    eligible_embeddings = corpus_embeddings[eligible_indices]
    scores = util.cos_sim(
        query_embedding, eligible_embeddings
    )[0]
    top = scores.topk(
        k=min(top_k, len(eligible_indices))
    )
```

3 評価結果の生データ

No.	期待カテゴリ	1 位カテゴリ	Top-3
1	競技 FPS	競技 FPS	正解
2	ゲーム+動画編集	ゲーム+動画編集	正解
3	生成 AI	生成 AI	正解
4	WQHD ゲーム	WQHD ゲーム	正解
5	4K ゲーム	4K ゲーム	正解
6	ゲーム配信	ゲーム配信	正解
7	3D 制作	3D 制作	正解
8	学生向け	学生向け	正解
9	シミュレーション	シミュレーション	正解
10	VR ゲーム	VR ゲーム	正解